

Aula 19 — Docker e Deploy

Módulo: Projeto, DevOps e Mercado

Carga horária: 4 horas

Projeto base: Full Stack (API .NET + React ou Blazor)

Objetivo da aula:

Compreender **containers**, criar **Dockerfile** para backend e frontend, orquestrar com **Docker Compose**, executar localmente e aplicar **versionamento** e **CI/CD** básico.

PARTE 1 — TEORIA (≈ 2 horas)

1 Containers (Conceito)

◆ O que são?

Containers empacotam **aplicação + dependências**, garantindo:

- Consistência entre ambientes
- Portabilidade
- Inicialização rápida

Diferença chave:

- **VM:** SO completo
 - **Container:** compartilha kernel
-

2 Dockerfile

◆ Função

Arquivo declarativo que define **como construir a imagem**.

Boas práticas:

- Multi-stage build
 - Imagens base oficiais
 - Camadas mínimas
 - Variáveis de ambiente
-

3 Docker Compose

◆ Função

Orquestra **múltiplos containers** (API, Front, DB).

Benefícios:

- `docker compose up` sobe tudo
 - Isolamento por serviço
 - Rede automática
-

4 CI/CD Básico (Visão Geral)

Pipeline mínimo:

```
Commit → Build → Test → Docker Build → Push Image → Deploy
```

Ferramentas comuns:

- GitHub Actions
 - GitLab CI
 - Azure DevOps
-

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Dockerizar **API .NET** e **Frontend**, orquestrar com **Docker Compose**, executar localmente e aplicar **versionamento**.

1 Dockerfile — API .NET (Multi-stage)

 src/Api/Dockerfile

```
# ----- Build ----- FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build WORKDIR /src # Copia csproj e restaura dependências
COPY Api/*.csproj Api/ RUN dotnet restore Api/Api.csproj # Copia o restante e publica COPY . . WORKDIR /src/Api RUN dotnet
publish -c Release -o /app/publish # ----- Runtime ----- FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime WORKDIR
/app EXPOSE 8080 # Copia artefatos publicados COPY --from=build /app/publish . # Variáveis de ambiente ENV
ASPNETCORE_URLS=http://+:8080 ENTRYPOINT ["dotnet", "Api.dll"]
```

- ✓ Multi-stage
 - ✓ Imagem final enxuta
 - ✓ Porta padronizada
-

2 Dockerfile — Frontend React

 frontend/Dockerfile

```
# ----- Build ----- FROM node:20-alpine AS build WORKDIR /app COPY package*.json ./ RUN npm install COPY . . RUN npm
run build # ----- Runtime ----- FROM nginx:alpine COPY --from=build /app/dist /usr/share/nginx/html EXPOSE 80 CMD
["nginx", "-g", "daemon off;"]
```

 Para **Blazor WASM**, o estágio final também é **nginx** com `wwwroot`.

3 Docker Compose — Orquestração

 docker-compose.yml

```
version: "3.9" services: api: build: context: . dockerfile: src/Api/Dockerfile image: aula19/api:1.0.0 ports: - "5001:8080"
environment: - ASPNETCORE_ENVIRONMENT=Production networks: - app-net frontend: build: context: ./frontend dockerfile: Dockerfile
image: aula19/frontend:1.0.0 ports: - "5173:80" depends_on: - api networks: - app-net networks: app-net: driver: bridge
```

- ✓ API e Front na mesma rede
 - ✓ Portas mapeadas
 - ✓ Dependências explícitas
-

4 Execução Local

```
docker compose build docker compose up
```

Acessos:

- Frontend → <http://localhost:5173>
- API → <http://localhost:5001/api/health> (exemplo)

Parar:

```
docker compose down
```

5 Versionamento de Imagens (SemVer)

Padrão recomendado:

```
MAJOR.MINOR.PATCH
```

Exemplos:

- 1.0.0 → primeira release
- 1.1.0 → feature nova
- 1.1.1 → correção

Aplicação no `docker-compose.yml` :

```
image: aula19/api:1.1.0
```

6 CI/CD Básico — GitHub Actions

 `.github/workflows/docker.yml`

```
name: CI - Build Docker Images on: push: branches: [ "main" ] jobs: build: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Build API run: docker build -t aula19/api:latest -f src/Api/Dockerfile . - name: Build Frontend run: docker build -t aula19/frontend:latest ./frontend
```

- ✓ Build automatizado
 - ✓ Pronto para push em registry
 - ✓ Base para deploy contínuo
-

7 Boas Práticas de Produção

- Variáveis sensíveis via `ENV/Secrets`
 - Logs em stdout
 - Health checks ativos
 - Imagens pequenas
 - Não rodar como root (quando possível)
-

✅ Resultado Esperado

- ✓ API e Front dockerizados
 - ✓ Orquestração com Docker Compose
 - ✓ Execução local consistente
 - ✓ Versionamento aplicado
 - ✓ Pipeline CI básico funcionando
-

📌 Atividade Avaliativa

👉 Desafio prático:

1. Adicionar banco (PostgreSQL) no Compose
2. Criar volume para persistência
3. Versionar imagens com tag do Git
4. Adicionar healthcheck no Dockerfile
5. Ajustar pipeline para push em registry